

Agentic On-Call Incident Response System

Multi-Agent LangGraph System with Production-Grade Failure Handling

Version 1.0 • July 2026
Classification: Internal — Development Team

Field	Value
Team size	5–7 developers
Timeline	2 weeks (10 working days)
LLM provider	OpenAI GPT-4o
Framework	LangGraph + LangChain
Versioning	Semantic (0.1.0 → 1.0.0)
Deliverable type	Portfolio / interview project

Table of Contents

1. Problem Statement

1.1 The Industry Problem

Production systems break at 2 AM. When they do, an on-call engineer is paged. They then spend the next 1–3 hours performing mechanical, repetitive steps: pulling logs from Datadog or CloudWatch, querying metrics dashboards, searching Confluence for relevant runbooks, cross-referencing recent deployments on GitHub, and trying fixes one by one. This process is exhausting, error-prone, and the number one cause of engineer burnout in SRE teams.

Current tools like PagerDuty, Datadog, and OpsGenie can detect problems and alert humans — but they cannot reason about what caused the problem, find the relevant runbook, or attempt a fix. They notify; they do not think.

1.2 What We Are Building

An autonomous multi-agent system that acts as a first responder to production incidents. When an alert fires, instead of immediately waking a human, the agent investigates first — reading logs, checking metrics, searching past incidents, identifying root causes, and either fixing the problem automatically or waking the engineer with a fully prepared diagnosis and recommended action.

The system uses LangGraph for orchestration, with three specialized subgraphs coordinating through a supervisor. The engineering challenge is not the happy path — it is making the system survive tool failures, avoid runaway loops, log every decision for post-mortems, pause for human approval on high-stakes actions, and resume from where it left off when something fails midway.

1.3 Why This Matters

For the industry: Reduces Mean Time To Resolution (MTTR) by automating the first 30–60 minutes of incident response. Prevents alert fatigue by only waking humans when genuinely needed.

For our portfolio: Demonstrates production-grade thinking — failure handling, observability, state machine design, and human-in-the-loop checkpoints — which is exactly what senior engineering roles evaluate for.

1.4 Key Differentiators (Novelty)

Standard agentic demos handle tool failures by counting retries. This system introduces four architectural contributions that signal production readiness:

1. Adaptive failure classification: Tool errors are classified semantically (timeout vs. rate limit vs. auth failure vs. malformed response) and routed to different recovery strategies — not just a blanket retry counter.

2. Confidence-gated human-in-the-loop: The agent computes a confidence score on its own output. Below a threshold, it pauses and presents its reasoning to a human reviewer — giving context, not just a raw output to approve blindly.

3. Dead letter queue for half-failed trajectories: When a run fails beyond recovery, the full state snapshot is serialized for later human review or automated retry — nothing is silently lost.

4. Audit trail as a first-class graph node: Every node appends a structured audit event. The trail serves as debugging artifact, post-mortem draft, and evaluation ground truth simultaneously.

2. Requirements

2.1 Functional Requirements

FR-1: Alert ingestion and state initialization

The system shall accept a structured incident alert (JSON format containing service name, metric, threshold violation, and timestamp) and initialize a typed agent state object with all fields set to appropriate defaults.

FR-2: Automated diagnosis

The system shall pull recent logs and current metrics for the alerting service, and produce a structured diagnosis identifying the failing component, blast radius, and severity classification (P0–P3).

FR-3: Root cause analysis

The system shall search a runbook knowledge base for matching incident patterns, check recent deployments for potential regression triggers, and produce a root cause hypothesis with an associated confidence score (0.0–1.0).

FR-4: Automated remediation with confidence gating

The system shall execute remediation steps from a matched runbook when confidence exceeds 0.85 and severity is not P0. For confidence between 0.50 and 0.85, the system shall pause for human review. Below 0.50 or for P0 incidents, the system shall escalate immediately.

FR-5: Tool failure handling

The system shall catch all tool call exceptions, classify the failure type (timeout, rate limit, auth, malformed, unknown), and route to the appropriate recovery strategy. Maximum 3 retries per tool with exponential backoff.

FR-6: Human-in-the-loop checkpoint

The system shall support pausing execution at defined checkpoints, presenting full diagnostic context to a human reviewer, and resuming based on human input. Implemented via LangGraph `interrupt_before` mechanism.

FR-7: Dead letter queue

The system shall serialize failed trajectory state snapshots to persistent storage when all recovery options are exhausted, with sufficient context for later human review or automated retry.

FR-8: Audit trail

Every node shall append a structured audit event recording: timestamp, node name, action taken, tool used, input summary, output summary, decision made, errors encountered, and confidence score where applicable.

FR-9: Incident resolution or escalation

Every incident shall terminate in one of three states: resolved (fix succeeded), escalated (human notified with full context), or dead-lettered (stored for recovery). No incident shall be silently dropped.

2.2 Non-Functional Requirements

NFR-1: Observability

All graph executions shall be traceable via LangSmith. Every node transition, tool call, and decision point shall appear in the trace.

NFR-2: Deterministic control flow

All routing decisions shall be based on explicit state field values, not implicit LLM reasoning. Conditional edges shall read typed fields (retry_count, confidence, severity) to determine the next node.

NFR-3: Runaway loop protection

A global kill switch (MAX_TOTAL_STEPS = 20) shall terminate any execution that exceeds the step limit, routing to the dead letter queue with full state preservation.

NFR-4: Graceful degradation

If one tool fails but others succeed, the agent shall continue with partial data rather than aborting entirely. The LLM shall reason over whatever data is available.

NFR-5: Checkpoint-based recovery

Agent state shall be checkpointed after every node via SQLite checkpointer, enabling resumption from the last successful step after a crash.

NFR-6: Evaluation readiness

The system shall include a test suite of 20+ synthetic incidents covering auto-resolve, escalation, and tool-failure scenarios, with computed metrics reported in the project README.

2.3 Technical Requirements

Component	Technology	Version
Language	Python	3.11+
Orchestration	LangGraph	≥ 0.2.0
LLM framework	LangChain	≥ 0.3.0
LLM provider	OpenAI GPT-4o	Latest
State validation	Pydantic	≥ 2.0
Tracing	LangSmith	≥ 0.1.0
Checkpointing	langgraph-checkpoint-sqlite	≥ 0.1.0
Notification (prod)	Gmail MCP	Latest
Version control	Git with semantic versioning	—

3. Expectations

3.1 Definition of Done

The project is considered complete when all of the following criteria are met:

Minimum viable deliverable (must ship)

1. Three subgraphs (diagnosis, root cause, remediation) wired through a supervisor graph, each calling at least one tool.
2. Adaptive failure classification on every tool-calling node — errors classified by type, recovery strategy selected accordingly.
3. Confidence-gated HITL checkpoint in the remediation subgraph with LangGraph interrupt_before.
4. Dead letter queue capturing failed trajectories with full state snapshots.
5. Audit trail node appending structured events at every state transition.
6. At least one real MCP tool (Gmail) for escalation notifications in production mode.
7. SQLite checkpointer enabling state persistence and crash recovery.
8. LangSmith tracing enabled and demonstrable.
9. 20+ synthetic test incidents with evaluation metrics computed and documented.
10. Professional README with architecture diagram, metrics table, and setup instructions.

Stretch goals (ship if time permits)

- S1. Trajectory summarization — compress long execution histories to prevent context window overflow during extended runs.
- S2. Self-healing supervisor — monitor subgraph health metrics and reassign tasks if a subgraph consistently fails.

3.2 Quality Expectations

Every pull request must meet these standards before merge:

Code: Type hints on all function signatures. Docstrings on every node function explaining what it reads from state, what it writes, and what decisions it makes.

Testing: Every node must be testable in isolation before graph wiring. Each subgraph must pass its test incidents independently before supervisor integration.

State discipline: Nodes return only the fields they modify — never the entire state object. No undocumented fields added to state without updating the schema.

Audit completeness: Every node appends exactly one audit event before returning. No node exits without logging.

3.3 Portfolio Presentation Expectations

This is a portfolio project. The README is as important as the code. It must contain:

Architecture section: Visual diagram showing the three subgraphs, their connections, and the failure handling flow.

Novelty section: Explicit statement of what this system does differently from standard agentic demos, naming each contribution.

Evaluation section: Table of metrics (resolution rate, escalation precision, failure recovery rate, audit completeness) computed over the test suite.

Demo section: CLI commands and expected output showing the system handling a normal incident, a tool failure, and an escalation.

4. Scope of Work

4.1 Scope Boundary — What Is Locked vs. Flexible

The team has freedom to adjust low-level implementation details but must preserve the high-level architectural decisions documented here.

Locked (cannot be changed without project lead approval)

- Three-subgraph architecture: Diagnosis → Root Cause → Remediation, orchestrated by a supervisor.
- Four mandatory novelty features: adaptive failure classification, confidence-gated HITL, dead letter queue, and audit trail.
- LangGraph as orchestration framework with typed state (TypedDict or Pydantic).
- OpenAI GPT-4o as the LLM provider.
- At least one real MCP tool (Gmail) in production mode.
- Semantic versioning with tagged releases at each phase completion.
- 20+ test incidents with documented evaluation metrics.

Flexible (team can adjust)

- Internal node names, function signatures, and file organization within each subgraph.
- Number and structure of nodes within each subgraph (e.g., combining pull_logs and pull_metrics into a single node is acceptable).
- Prompt templates and LLM interaction patterns (structured output parsing, few-shot examples, etc.).
- Specific retry backoff durations (1s/3s/10s is a recommendation, not a mandate).
- Mock data content — team can create their own incidents, runbooks, and deployment histories.
- Additional tools beyond the required three, including which real MCP tools to add beyond Gmail.
- Testing approach (pytest, unittest, manual scripts) — as long as each node is tested in isolation.
- README styling and documentation format — as long as all required sections are present.

4.2 Out of Scope

The following are explicitly excluded from this project:

- Frontend / web UI — CLI and notebook-based demonstration is sufficient.
- Real production deployment — local execution with clear architecture is the target.
- Real Datadog / CloudWatch integration — mock observability tools are acceptable.
- Multi-tenant support or user authentication.

- CI/CD pipeline setup — manual build and run is acceptable.
- Performance benchmarking or latency optimization.

5. Work Items

5.1 Work Item Registry

Each work item is assigned a unique ID, priority level, estimated effort, and dependency chain. The team should complete items in dependency order within each priority tier.

ID	Work Item	Priority	Effort	Depends On	Owner
WI-01	Define IncidentState TypedDict/Pydantic schema	P0	0.5d	—	Unassigned
WI-02	Build MockLogAPI with 4 failure modes	P0	0.5d	WI-01	Unassigned
WI-03	Build MockRunbookSearch with keyword matching	P0	0.5d	WI-01	Unassigned
WI-04	Build MockGitHubAPI with deployment history	P0	0.5d	WI-01	Unassigned
WI-05	Build MockNotificationService	P0	0.25d	WI-01	Unassigned
WI-06	Build failure classifier (classify_failure + classify_response)	P0	0.5d	—	Unassigned
WI-07	Build audit trail utility (append, format, save)	P0	0.5d	WI-01	Unassigned
WI-08	Build dead letter queue utility (send, review)	P0	0.5d	WI-01	Unassigned
WI-09	Create config.py with all thresholds	P0	0.25d	—	Unassigned
WI-10	Create 3 seed test incidents (JSON)	P0	0.25d	—	Unassigned
WI-11	Build pull_logs node with failure handling	P0	0.5d	WI-02, WI-06	Unassigned
WI-12	Build pull_metrics node with failure handling	P0	0.5d	WI-02, WI-06	Unassigned
WI-13	Build analyze_diagnosis node (LLM call)	P0	1d	WI-11, WI-12	Unassigned
WI-14	Build handle_diagnosis_failure node	P0	0.5d	WI-06, WI-08	Unassigned
WI-15	Wire diagnosis subgraph with conditional edges	P0	0.5d	WI-11 to WI-14	Unassigned
WI-16	Test diagnosis subgraph in isolation	P0	0.5d	WI-15	Unassigned
WI-17	Build search_runbooks node with failure handling	P0	0.5d	WI-03, WI-06	Unassigned
WI-18	Build check_deployments node with failure handling	P0	0.5d	WI-04, WI-06	Unassigned
WI-19	Build analyze_root_cause node (LLM call)	P0	1d	WI-17, WI-18	Unassigned
WI-20	Wire root cause subgraph with conditional edges	P0	0.5d	WI-17 to WI-19	Unassigned
WI-21	Test root cause subgraph in isolation	P0	0.5d	WI-20	Unassigned
WI-22	Build evaluate_confidence decision node	P0	0.5d	WI-09	Unassigned
WI-23	Build execute_fix and verify_fix nodes	P0	1d	WI-22	Unassigned

ID	Work Item	Priority	Effort	Depends On	Owner
WI-24	Build human_review HITL checkpoint node	P0	1d	WI-22	Unassigned
WI-25	Build escalate and close_incident nodes	P0	0.5d	WI-05, WI-07	Unassigned
WI-26	Wire remediation subgraph (3-branch routing)	P0	1d	WI-22 to WI-25	Unassigned
WI-27	Test remediation subgraph — all 3 branches	P0	0.5d	WI-26	Unassigned
WI-28	Build supervisor graph wiring 3 subgraphs	P0	1d	WI-16, WI-21, WI-27	Unassigned
WI-29	Add kill switch node (MAX_TOTAL_STEPS)	P0	0.25d	WI-28	Unassigned
WI-30	Add SQLite checkpointer for crash recovery	P1	0.5d	WI-28	Unassigned
WI-31	Enable LangSmith tracing	P1	0.5d	WI-28	Unassigned
WI-32	Integrate Gmail MCP for real escalation emails	P1	1d	WI-25	Unassigned
WI-33	End-to-end test: all 3 seed incidents	P0	0.5d	WI-28 to WI-32	Unassigned
WI-34	Create 17 additional synthetic incidents	P1	1d	WI-10	Unassigned
WI-35	Build evaluation runner (run_eval.py)	P1	1d	WI-33	Unassigned
WI-36	Build metrics calculator (metrics.py)	P1	0.5d	WI-35	Unassigned
WI-37	Run full evaluation, document results	P1	0.5d	WI-36	Unassigned
WI-38	Write professional README with all sections	P0	1d	WI-37	Unassigned
WI-39	Trajectory summarization node	P2	1.5d	WI-28	Unassigned
WI-40	Self-healing supervisor logic	P3	2d	WI-28	Unassigned

5.2 Priority Legend

Priority	Meaning	Must ship?
P0	Critical — project fails without this	Yes, non-negotiable
P1	High — significantly weakens portfolio without this	Yes, strongly expected
P2	Stretch — impressive if included, acceptable if omitted	No, time-permitting
P3	Aspirational — pursue only if ahead of schedule	No

5.3 Effort Summary

Category	Work items	Total effort	Developers needed
Foundation (WI-01 to WI-10)	10	4 person-days	2 developers in parallel
Diagnosis subgraph (WI-11 to WI-16)	6	3.5 person-days	1–2 developers

Category	Work items	Total effort	Developers needed
Root cause subgraph (WI-17 to WI-21)	5	3 person-days	1-2 developers
Remediation subgraph (WI-22 to WI-27)	6	4.5 person-days	2 developers
Supervisor + production (WI-28 to WI-33)	6	3.75 person-days	1-2 developers
Evaluation + docs (WI-34 to WI-38)	5	4 person-days	1-2 developers
Stretch (WI-39 to WI-40)	2	3.5 person-days	1 developer

Total mandatory effort: ~22.75 person-days. With 5 developers over 10 working days (50 person-days available), this provides ~55% buffer for debugging, integration issues, and unforeseen complexity.

6. Implementation Phases & Versioning

The project is divided into five phases. Each phase ends with a tagged Git release. No phase begins until the previous phase's release criteria are met. The team should treat each release as a checkpoint — if the project must stop early, the latest tagged version is the deliverable.

6.1 Phase 1 — Foundation

Field	Detail
Release tag	v0.1.0
Duration	Days 1–2
Work items	WI-01 through WI-10
Developers	2–3 (parallelizable)

Deliverables

- `state/schemas.py` — complete `IncidentState` TypedDict with all five field categories (input, output, control, routing, logging), `AuditEvent` Pydantic model, `DeadLetterEntry` Pydantic model, and `create_initial_state()` factory function.
- `tools/` — four mock tools (`MockLogAPI`, `MockRunbookSearch`, `MockGitHubAPI`, `MockNotificationService`), each with at least 3 configurable failure modes (timeout, rate_limit, empty, malformed).
- `utils/` — `failure_classifier.py` (`classify_failure` + `classify_response`), `audit_trail.py` (`append_audit_event`, `format_trail_for_human`, `save_trail_to_file`), `dead_letter_queue.py` (`send_to_dlq`, `review_dlq`).
- `config.py` — all thresholds (retry limits, confidence gates, severity rules, paths).
- `data/incidents/` — 3 seed incidents covering auto-resolve, auto-resolve (different root cause), and forced escalation.
- `main.py` — CLI entry point with `--test-tools`, `--test-state`, and `--review-dlq` commands.

Release criteria for v0.1.0

- ✓ `python main.py --test-tools` passes — all 4 tools work normally and all failure modes raise correct exceptions.
- ✓ `python main.py --test-state` passes — state creation works, all fields present with correct defaults.
- ✓ `classify_failure` correctly maps: `TimeoutError` → `timeout`, `429` → `rate_limit`, `401` → `auth`, `unknown` → `unknown`.
- ✓ `classify_response` correctly distinguishes: `ok`, `empty` (not an error), `malformed` (is an error).
- ✓ Audit trail appends, formats, and saves to file correctly.
- ✓ Dead letter queue writes and reads JSON files correctly.
- ✓ Git tag `v0.1.0` created.

6.2 Phase 2 — Individual Subgraphs

Field	Detail
Release tag	v0.2.0
Duration	Days 3–6
Work items	WI-11 through WI-27
Developers	3–4 (subgraphs built in parallel)

Parallelization strategy

The three subgraphs can be built simultaneously by different developers since they share the state schema (locked in v0.1.0) but do not depend on each other. Assign 1–2 developers per subgraph.

Stream	Subgraph	Work items	Developers
Stream A	Diagnosis agent	WI-11 to WI-16	Dev 1 + Dev 2
Stream B	Root cause agent	WI-17 to WI-21	Dev 3 + Dev 4
Stream C	Remediation agent	WI-22 to WI-27	Dev 5 + Dev 6

Deliverables

- `agents/diagnosis.py` — `pull_logs`, `pull_metrics`, `analyze_diagnosis`, `handle_diagnosis_failure` nodes wired into a compiled `StateGraph`. Each tool-calling node wraps calls in `try/except`, classifies failures, and appends audit events.
- `agents/root_cause.py` — `search_runbooks`, `check_deployments`, `analyze_root_cause` nodes with failure handling, wired into a compiled `StateGraph`.
- `agents/remediation.py` — `evaluate_confidence`, `execute_fix`, `verify_fix`, `human_review`, `escalate`, `close_incident` nodes with three-branch conditional routing (auto-fix / HITL / escalate).

Critical implementation notes

Every tool-calling node: must follow the pattern — try tool call → on exception: `classify_failure()` → update error state → audit → return. On success: `classify_response()` → update data state → audit → return.

Graceful degradation: if logs fail but metrics succeed (or vice versa), the diagnosis subgraph must still proceed to `analyze_diagnosis` with partial data. Only if ALL data sources fail should it route to failure handling.

Confidence gating: the `evaluate_confidence` node must read `root_cause_confidence` AND severity to make its routing decision. P0 incidents never auto-resolve regardless of confidence.

Release criteria for v0.2.0

- ✓ Each subgraph compiles and runs independently against seed incidents.
- ✓ Diagnosis subgraph produces `diagnosis_summary` and severity for incident_001.
- ✓ Root cause subgraph produces `root_cause_hypothesis` and confidence for incident_001.
- ✓ Remediation subgraph correctly routes: high confidence → auto-fix, low confidence → escalate.

- ✓ Tool failure injection (timeout mode on any tool) triggers failure classification and retry — not a crash.
- ✓ Audit trail contains entries from every node that executed.
- ✓ Git tag v0.2.0 created.

6.3 Phase 3 — Supervisor Integration & Production Hardening

Field	Detail
Release tag	v0.3.0
Duration	Days 7–9
Work items	WI-28 through WI-33
Developers	2–3 (integration is sequential)

Deliverables

- agents/supervisor.py — full supervisor graph wiring all three subgraphs with phase-based routing, kill switch node, and global error handling.
- SQLite checkpointer integrated — state persisted after every node, enabling crash recovery and HITL pause/resume.
- LangSmith tracing enabled — every node, tool call, and conditional edge visible in the LangSmith dashboard.
- Gmail MCP integration — real escalation emails sent to a configured address when the agent escalates.
- End-to-end test passing on all 3 seed incidents: INC-001 (auto-resolve), INC-002 (auto-resolve), INC-003 (escalate).

Supervisor wiring approach

Start with Approach B (flat graph with phase routing) — all nodes in one graph, state["current_phase"] determines which section runs. Once this works end-to-end, refactor to Approach A (compiled subgraphs as nodes) if time permits. Both are valid; Approach B is faster to build and debug.

Release criteria for v0.3.0

- ✓ python main.py runs incident_001 end-to-end: alert → diagnosis → root cause → auto-fix → resolved.
- ✓ python main.py data/incidents/incident_003.json routes to escalation — human_review HITL checkpoint fires.
- ✓ Kill switch triggers when total_steps exceeds 20 — state routed to DLQ, not infinite loop.
- ✓ Injecting tool failures on any single tool does not crash the pipeline — failure is classified and handled.
- ✓ LangSmith trace shows full node-by-node execution for at least one incident.
- ✓ Gmail MCP sends a real email on escalation (or documented mock fallback if MCP unavailable).
- ✓ Audit trail files generated in data/audit_trail/ for every run.

✓ Git tag v0.3.0 created.

6.4 Phase 4 — Evaluation & Documentation

Field	Detail
Release tag	v1.0.0
Duration	Days 9–10
Work items	WI-34 through WI-38
Developers	2–3 (evaluation + docs in parallel)

Deliverables

- data/incidents/ — 20 total synthetic incidents covering: 10 auto-resolvable (known root cause, matching runbook), 5 escalation-required (unknown issue, no runbook match), 5 with tool failures injected (test recovery paths).
- evaluation/run_eval.py — script that runs all 20 incidents through the agent and collects results.
- evaluation/metrics.py — computes: resolution rate, correct escalation rate (precision + recall), tool failure recovery rate, audit trail completeness score, average steps to resolution.
- README.md — professional portfolio README containing: project overview, architecture diagram, novelty section, setup instructions, CLI usage, evaluation metrics table, demo output, and tech stack.

Evaluation metrics to compute

Metric	Formula	Target
Resolution rate	Auto-resolved / Total resolvable incidents	> 80%
Escalation precision	Correct escalations / Total escalations	> 85%
Escalation recall	Correct escalations / Incidents needing escalation	> 90%
Failure recovery rate	Gracefully handled failures / Total injected failures	> 80%
Audit completeness	Events logged / Total node executions	100%
DLQ capture rate	Captured failed runs / Total unrecoverable failures	100%
Avg. steps to resolution	Sum of steps / Total resolved incidents	< 12

Release criteria for v1.0.0

- ✓ 20 incidents run through the agent with results logged.
- ✓ All metrics computed and documented in README.
- ✓ README contains all required sections (architecture, novelty, evaluation, demo, setup).

- ✓ A clean git log with meaningful commit messages across all phases.
- ✓ Git tag v1.0.0 created. This is the portfolio-ready release.

6.5 Phase 5 — Stretch Goals (If Time Permits)

Field	Detail
Release tag	v1.1.0
Duration	Remaining time after v1.0.0
Work items	WI-39, WI-40
Developers	1–2 (independent features)

Stretch goal A: Trajectory summarization (WI-39)

Add a summarizer node that activates every N steps (configurable, default 5). It reads the last N audit events, generates a compressed summary via LLM, writes it to state["trajectory_summary"], and prunes raw step history. This prevents context window overflow on long-running incidents.

Stretch goal B: Self-healing supervisor (WI-40)

The supervisor tracks per-subgraph health metrics (failure_rate, avg_steps). If a subgraph fails twice on the same incident, the supervisor reassigns the task to a simplified fallback path (direct escalation with whatever data is available). Log the reassignment with reasoning in the audit trail.

Release criteria for v1.1.0

- ✓ Stretch features work without breaking any v1.0.0 functionality.
- ✓ Updated README documents the stretch features.
- ✓ Git tag v1.1.0 created.

7. Version Summary & Release Timeline

Version	Phase	Days	Key milestone
v0.1.0	Foundation	1–2	State schema, tools, utilities — all tested
v0.2.0	Subgraphs	3–6	Three subgraphs working independently
v0.3.0	Integration	7–9	Full pipeline end-to-end with production features
v1.0.0	Evaluation	9–10	Portfolio-ready with metrics and documentation
v1.1.0	Stretch	Bonus	Trajectory summarization + self-healing supervisor

7.1 Critical Path

The critical path through the project is: WI-01 (state schema) → WI-11/WI-17/WI-22 (first nodes of each subgraph, parallelizable) → WI-15/WI-20/WI-26 (subgraph wiring) → WI-28 (supervisor) → WI-33 (end-to-end test) → WI-38 (README). Any delay on this path delays the final deliverable.

7.2 Risk Mitigation

Risk: LLM output parsing is fragile and produces unexpected formats.

Mitigation: Use structured output prompting with explicit JSON schemas. Add try/except around all LLM response parsing with fallback to raw text.

Risk: Gmail MCP integration hits authentication issues.

Mitigation: Build the notification node to accept either real MCP or mock tool via config flag. If MCP fails, demo with mock and document the MCP approach.

Risk: Subgraph wiring produces state conflicts when integrated.

Mitigation: All subgraphs share the same IncidentState schema locked in v0.1.0. No subgraph may add fields without updating the shared schema.

Risk: Team finishes ahead of schedule.

Mitigation: Stretch goals (WI-39, WI-40) are designed and scoped. LLM-as-judge evaluation (quality scoring of agent reasoning) is an additional stretch target.

— End of Document —